

Clase 244 — Políticas como código con OPA

Parte: **11 — DevSecOps y seguridad del SDLC** · Fuente: Documentación de Open Policy Agent (OPA) y Agile Application Security (gobierno de seguridad automatizado) 🕒 Duración estimada: **110 min** · Nivel: **Avanzado**

🎯 Objetivo

Expresar las reglas de seguridad y cumplimiento como **código versionable, testeable y auditable** en vez de como documentos o revisiones manuales. Con Open Policy Agent (OPA) y su lenguaje Rego escribiremos políticas que validan configuración de infraestructura (Terraform, Kubernetes), Dockerfiles y pipelines, y las integraremos como gate en CI con **Conftest**. "Policy as Code" hace que el cumplimiento sea automático, consistente y no dependa de que alguien recuerde revisarlo.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** qué es policy as code y por qué desacopla decisión de aplicación (PDP/PEP).
- 2. **Escribir** políticas en Rego con reglas `deny` / `allow` y mensajes claros.
- 3. **Evaluar** manifiestos (Kubernetes, Terraform plan, Dockerfile) con Conftest.
- 4. **Testear** las políticas con casos positivos y negativos.
- 5. **Integrar** las políticas como gate obligatorio en el pipeline.

📖 Temas

#	Tema	Por qué importa
1	Policy as Code: motivación	Cumplimiento automático, consistente y auditable
2	OPA y el modelo PDP/PEP	Separar quién decide de quién aplica
3	Rego básico	Lenguaje declarativo de las políticas
4	Conftest	Aplicar OPA a archivos de configuración en CI
5	Casos: K8s, Terraform, Docker	Dónde aporta valor
6	Testear políticas	Las políticas también necesitan tests
7	Gatekeeper (admisión K8s)	OPA en el clúster en runtime

📖 Definiciones y características

- **Policy as Code:** expresar reglas como código versionado y probado. *Característica:* revisable en PRs, con historial y tests, no en un PDF.
- **OPA:** motor de políticas de propósito general. *Característica:* recibe input JSON y una consulta, devuelve una decisión.

- **Rego:** lenguaje declarativo de OPA. *Característica:* se centra en describir qué es válido/denegado, no en el cómo.
- **PDP/PEP:** Policy Decision Point (decide) y Policy Enforcement Point (aplica). *Característica:* OPA es el PDP; el pipeline o el clúster es el PEP.
- **Conftest:** herramienta que usa OPA para validar archivos de configuración. *Característica:* ideal como gate de CI sobre YAML/JSON/HCL/Dockerfile.
- **Gatekeeper:** controlador de admisión de Kubernetes basado en OPA. *Característica:* aplica políticas en runtime al crear recursos.

Herramientas y preparación

- **OPA** (binario `opa`) para evaluar y testear Rego.
- **Conftest** para validar configuración en el pipeline.
- **OPA Gatekeeper** (opcional) para admisión en Kubernetes.

```
# Instalar (ejemplos):
brew install opa conftest      # macOS
# o descarga los binarios de releases

opa version && conftest --version
```

Laboratorio guiado

1. **Escribe una política que prohíba contenedores privilegiados** en Kubernetes. Crea `policy/security.rego`:

```
package main

deny[msg] {
  input.kind == "Deployment"
  c := input.spec.template.spec.containers[_]
  c.securityContext.privileged == true
  msg := sprintf("El contenedor '%s' no puede ser privileged", [c.name])
}

deny[msg] {
  input.kind == "Deployment"
  c := input.spec.template.spec.containers[_]
  not c.resources.limits
  msg := sprintf("El contenedor '%s' debe declarar resource limits", [c.name])
}
```

2. **Evalúa un manifiesto** con Conftest:

```
conftest test deployment.yaml -p policy/
```

Debe fallar si el deployment es privilegiado o carece de limits. 3. **Testea la política.** Crea `policy/security_test.rego` con un input que debe pasar y otro que debe denegar; ejecútalos:

```
opa test policy/ -v
```

4. **Política para Dockerfile.** Escribe una regla que deniegue `USER root` o la ausencia de instrucción `USER` y válidala con Conftest sobre el Dockerfile parseado.
5. **Política para Terraform.** Ejecuta `terraform plan -out tfplan && terraform show -json tfplan > plan.json` y escribe una política que deniegue buckets S3 públicos; valida con Conftest.
6. **Integra el gate en CI.** Añade un job que corra `conftest test` sobre los manifiestos y falle el build ante cualquier `deny`.
7. **(Opcional) Gatekeeper.** Despliega una ConstraintTemplate equivalente en un clúster de práctica para aplicar la política en runtime.

Ejercicios

1. Escribe una política que exija que todo Deployment corra como no-root.
2. Añade una regla que prohíba imágenes con tag `:latest`.
3. Crea tests positivos y negativos para tus políticas con `opa test`.
4. Valida un `terraform plan` en JSON con una política de red.
5. Integra Conftest como gate obligatorio en un pipeline.
6. Traduce una de tus políticas a una Constraint de Gatekeeper.

Reto verificable

Implementa un conjunto de políticas como código con tests y gate en CI.

Criterio de aceptación: (a) existen al menos tres políticas Rego que validan configuración real (K8s/Terraform/Dockerfile); (b) cada política tiene tests positivos y negativos que pasan con `opa test`; (c) Conftest corre como gate en CI y rompe el build ante violaciones; y (d) los mensajes de `deny` son claros y accionables para quien los recibe.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
La política no deniega nada	El <code>input</code> no tiene la forma esperada. Inspecciona con <code>conftest parse</code> o <code>opa eval</code> .
Rego lanza "multiple assignments"	Reusaste una variable con <code>=</code> . Usa <code>:=</code> para asignación y nombres únicos.
Falsos denies en recursos válidos	Regla demasiado amplia. Acota con condiciones de <code>kind</code> /campos y añade tests.
Nadie entiende por qué falló el gate	Mensajes de <code>deny</code> genéricos. Usa <code>sprintf</code> con nombre del recurso y razón.
La política pasa en local pero no en CI	Distinto path de políticas o formato de input. Alinea el comando <code>conftest</code> y el parseo.

? Preguntas frecuentes

? **¿Conftest y Gatekeeper hacen lo mismo?** Comparten OPA/Rego, pero Conftest valida archivos en el pipeline (shift-left) y Gatekeeper aplica políticas en el clúster en runtime (admisión). Se complementan: defensa en profundidad.

? **¿Rego es difícil de aprender?** Tiene una curva por ser declarativo, pero para políticas de seguridad el patrón `deny[msg] { condiciones }` cubre la mayoría de casos. Empieza copiando ejemplos de la librería de la comunidad.

? **¿Debo escribir todas las políticas desde cero?** No. Existen librerías como las de Conftest y las de Gatekeeper con políticas comunes (CIS, buenas prácticas) que puedes adaptar.

? **¿Policy as code reemplaza al equipo de seguridad?** No: codifica sus decisiones para aplicarlas de forma consistente y automática. Seguridad define la política; el código la aplica en cada cambio.

🔗 Referencias

- Open Policy Agent — <https://www.openpolicyagent.org/docs/latest/>
- Conftest — <https://www.conftest.dev/>
- OPA Gatekeeper — <https://open-policy-agent.github.io/gatekeeper/>
- Rego Playground — <https://play.openpolicyagent.org/>
- CNCF Policy as Code — <https://www.cncf.io/blog/2021/09/policy-as-code/>

➡ Siguiente clase

Clase 245 - Gestion de vulnerabilidades a escala